

Modern Cloud Computing

Lecture 10: Resource Virtualization I

Chongzhou Fang

Rochester Institute of Technology

Today's Topics

- Basic Concepts
- The Popek/Goldberg Theorem
- Virtualization without architectural support
- I/O virtualization
 - w/ HW Support
 - w/o HW Support

E. Bugnion, J. Nieh, D. Tsafirir, *Hardware and Software Support for Virtualization*, Synthesis Lectures on Computer Architecture #38, Morgan & Claypool, 2017.

Basic Concepts

What Is Virtualization?

Virtualization is the application of the *layering* principle through *enforced modularity*, whereby the exposed virtual resource is **identical** to the underlying physical resource being virtualized.

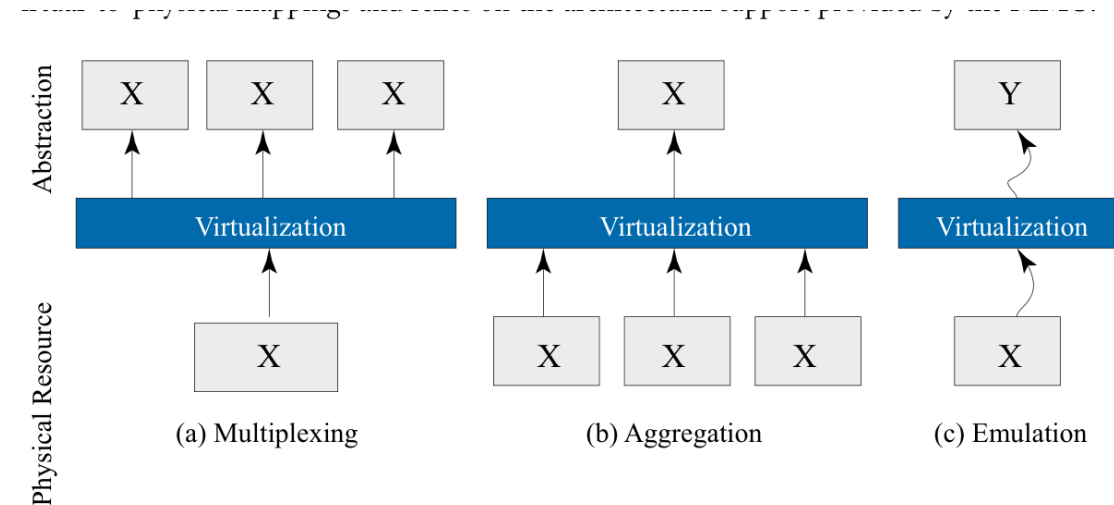
Two ingredients:

- **Layering**: add a level of indirection over a single lower layer, exposing a well-defined namespace
- **Enforced modularity**: clients *cannot* bypass the layer to touch the physical resource

The abstraction is **compatible** with the resource, so existing software works unchanged (e.g., RAID exposes a disk to the filesystem).

Three Implementation Techniques

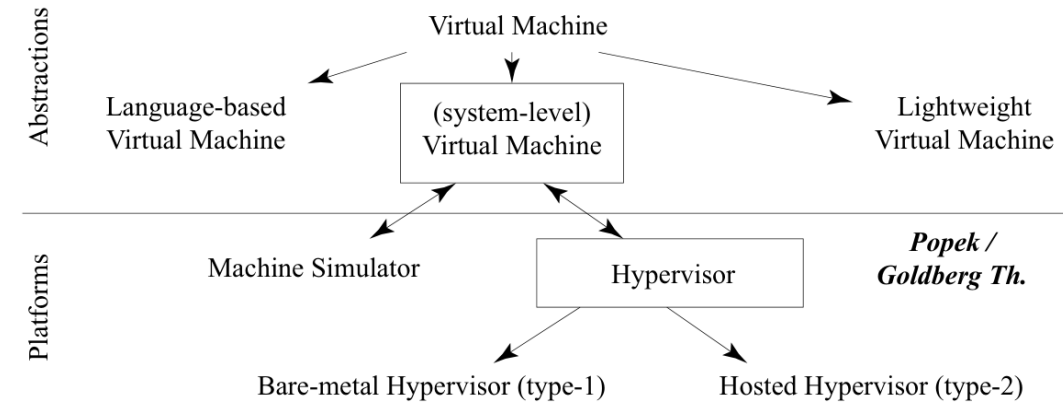
- **Multiplexing** — one physical resource, many virtual ones
 - *In space*: pages of physical memory across address spaces
 - *In time*: the OS scheduler time-slices a core among threads
- **Aggregation** — many physical resources exposed as one (RAID volume; DIMMs → one physical address space)
- **Emulation** — software indirection exposes a resource that isn't physically there (Rosetta emulating PowerPC on x86; a RAM disk)



What Is a Virtual Machine?

A **virtual machine** is a complete compute environment with its own isolated processing capabilities, memory, and communication channels.

- **Language-based VMs** — JVM, CLR, JS engines (out of scope)
- **Lightweight VMs** — hardware+software isolation for native code: Denali, Native Client, Vx32, containers (Docker, FreeBSD Jail)
- **System-level VMs** — an entire computer, running a commodity **guest operating system** (*our focus*)



Hypervisors vs. Machine Simulators

A **hypervisor** runs system-level VMs while minimizing execution overhead, using **direct execution** plus **trap-and-emulate**.

A **machine simulator** is a user-level application that accurately simulates the architecture — 5× to 1000× slowdown.

Popek and Goldberg's three criteria for a VMM:

- **Equivalence** — the VM is a *duplicate* of the real machine
- **Safety** — the VMM is in complete control, assuming *nothing* about guest software
- **Performance** — "at worst only minor decreases in speed"

The **performance** criterion is exactly what separates hypervisors from simulators.

Trap-and-Emulate

Hand off sensitive operations to the hypervisor

- VMM emulates the effect of these operations on virtual hardware provided to the guest OS
- VMM controls how the VM interacts with physical hardware
- VMM fools the guest OS into thinking that it runs at the highest privilege level

Performance implications

- Almost no overhead for non-sensitive instructions
- Large overhead for sensitive instructions

(Source: [UCSD CSE291J-Winter 20 slides](#))

Type-1 and Type-2 Hypervisors

Type-1 (bare-metal)

- The VMM runs on a bare machine
- It performs scheduling and *real* resource allocation itself
- VMware ESX Server, Xen, Microsoft Hyper-V

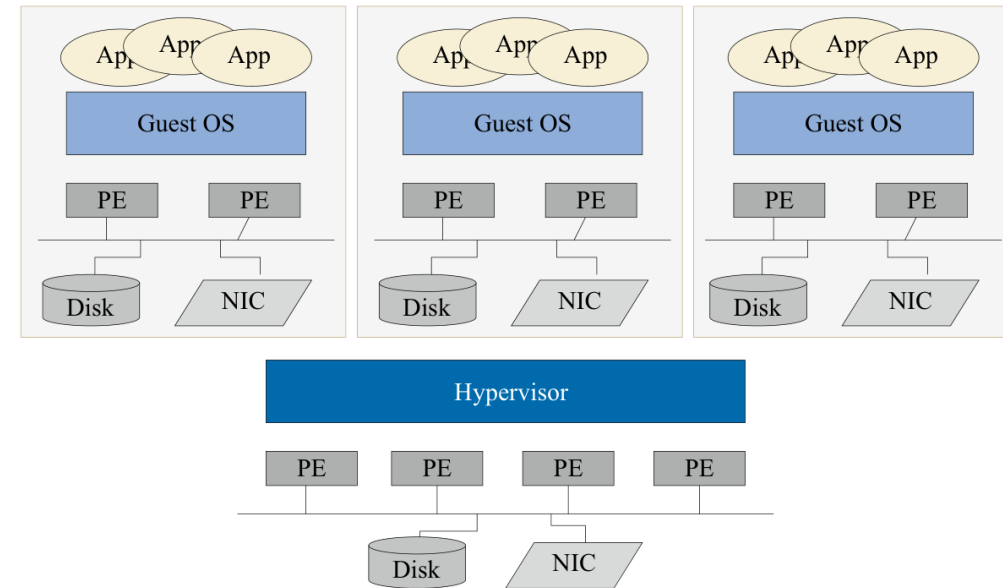
Type-2 (hosted)

- The VMM runs on an *extended host*, under a host OS
- The host OS does resource allocation and scheduling
- VMware Workstation/Fusion, KVM, VirtualBox, Parallels

A Sketch Hypervisor

In its most basic form, a hypervisor:

- **Multiplexes** the processing elements (CPU + physical memory) across VMs
 - Scheduling task $\Rightarrow$ **direct execution** at reduced privilege
 - Privileged guest instructions trap $\Rightarrow$ **trap-and-emulate**
- **Emulates** everything else — the I/O bus and I/O devices



Three Approaches to Virtualization

- **Full (software) virtualization** — maximize hardware compatibility on architectures *lacking* support; runs unmodified guests (VMware, §3.2)
- **Hardware virtualization (HVM)** — architectural support exists; rely *exclusively* on direct execution; unmodified guests (VT-x, ARM VE)
- **Paravirtualization** — trade equivalence for simplicity/efficiency; originally required a *modified guest kernel* (Denali, early Xen); today used to *augment* HVM (e.g., high-performance front-end devices)

The Popek/Goldberg Theorem

The Popek/Goldberg Theorem

Popek & Goldberg, "*Formal Requirements for Virtualizable Third-Generation Architectures*," CACM 1974.

- Determines whether a given **ISA** can be virtualized by a VMM using **multiplexing** (i.e., direct execution + trap-and-emulate)
- If the hypothesis holds, *any* OS that runs on the bare hardware also runs, **unmodified**, inside a VM
- Motivated by architectures (DEC PDP-10) that *accidentally* broke virtualization
- Ignored by architects for decades; **Intel and AMD explicitly designed VT-x/AMD-V to satisfy it**

The Model: A "Third-Generation" Architecture

- One processor, two execution levels: **supervisor** ($M=s$) and **user** ($M=u$)
- Virtual memory via **segmentation only**: a single relocation-bound pair (B, L) ; valid virtual addresses $[0, L)$ map to $[B, B+L)$; no paging
- Physical memory contiguous from 0, size SZ known at reset
- System state = **processor status word** $PSW = (M, B, L, PC)$
- **Trap architecture**: save PSW to $MEM[0]$, load new PSW from $MEM[1]$
- At least one instruction can load (M, B, L, PC) into the PSW from memory
- I/O and interrupts ignored for simplicity

The Formal Research Question

Given a computer that meets this basic architectural model, under which precise conditions can a VMM be constructed, so that the VMM:

- can execute one or more virtual machines;
- is in complete control of the machine at all times;
- supports arbitrary, unmodified, potentially malicious operating systems designed for that same architecture; and
- is efficient enough to show at worst a small decrease in speed?

Answer $\Rightarrow$ the VM is an **efficient, isolated duplicate** of the real machine
(= **equivalence, safety, performance**).

Instruction Classification

- **Control-sensitive** — the instruction *can update* the system state
- **Behavior-sensitive** — its semantics *depend on* the actual system state (e.g., on `M`, or on the relocation registers)
- **Innocuous** — neither
- **Privileged** — executes only in supervisor mode; **traps** when attempted from user mode

Theorem 1

Theorem 1. For any conventional third-generation computer, a virtual machine monitor may be constructed if the set of **sensitive** instructions for that computer is a **subset** of the set of **privileged** instructions.

Corollary in practice: a VMM is really nothing more than an operating system — it lets an untrusted component run directly on the hardware and must stay in control. The difference is that it runs whole *virtual machines* rather than *applications*.

$$\{\text{control-sensitive}\} \cup \{\text{behavior-sensitive}\} \subseteq \{\text{privileged}\}$$

Recursive Virtualization

Theorem 2 — recursive virtualization support. The idealized architecture can also run a VMM *inside* a VM (guest-supervisor mode).

The theorem states the extra assumptions on the *VMM* (not the architecture) needed for this: the VMM itself must be able run unmodified inside a VM.

Hybrid Virtual Machines

Theorem 3. A **hybrid** VMM may be constructed for any conventional third-generation machine in which the set of **user-sensitive** instructions are a subset of the privileged instructions.

- Weaker than Theorem 1: Theorem 1 requires all sensitive instructions to be privileged, while Theorem 3 requires only user-sensitive instructions to be privileged because it interprets all guest kernel code.
 - User-sensitive instructions: are instructions that are control-sensitive or behavior-sensitive in at least one user-mode state; they may also be sensitive in supervisor mode.
 - Non-User-sensitive example: An instruction changes processor's mode when executing in supervisor mode, but does nothing in User mode)
- Hybrid VMM: direct execution of ordinary guest-user code, with sensitive operations trapped; 100% interpretation of guest-supervisor code.
- Performance: most attractive when little time is spent in the guest kernel. Related approaches reduce overhead through binary translation (rewriting guest machine code at runtime) or paravirtualization (modifying the guest OS to cooperate with the hypervisor).

Violation of Real ISAs

Three recurring ways real ISAs break the theorem:

- **Direct access to physical memory** — architecture hardcodes physical memory into portions of virtual memory (**MIPS**)
- **Location-sensitive instructions** — unprivileged instructions expose the location of sensitive data structures (x86 descriptor tables)
- **Control-sensitive violations** — the ISA defines *different semantics* for the same instruction depending on privilege level (x86 `iret` , `popf`)

MIPS: `KSEG0` / `KSEG1` are **unmapped**, hardcoded regions $\Rightarrow$ *location-sensitivity*. Any kernel compiled for them faults on every load/store when de-privileged $\Rightarrow$ virtualizing MIPS requires **recompiling the guest kernel**.

Region	Base	Length	Access K,S,U	MMU	Cache
USEG	0x0000 0000	2 GB	✓,✓ ✓	mapped	cached
KSEG0	0x8000 0000	512 MB	✓,x,x	unmapped	cached
KSEG1	0xA000 0000	512 MB	✓,x,x	unmapped	uncached
KSSEG	0xC000 0000	512 MB	✓,✓, x	mapped	cached
KSEG3	0xE000 0000	512 MB	✓,x,x	mapped	cached

Violation: x86-32

17 sensitive-but-unprivileged instructions.

Before VMware, Intel engineers were convinced x86 could not be practically virtualized.

v8086 mode *was* strictly virtualizable, but only for 16-bit VMs.

Group	Instructions
Access to interrupt flag	pushf, popf, iret
Visibility into segment descriptors	lar, verr, verw, lsl
Segment manipulation instructions	pop <seg>, push <seg>, mov <seg>
Read-only access to privileged state	sgdt, sldt, sidt, smsw
Interrupt and gate instructions	fcall, longjump, retfar, str, int <n>

Violation: ARM (pre-Virtualization Extensions)

24 sensitive-but-unprivileged instructions in ARMv6; similar ones persist through ARMv8.

Three problem classes:

- **Status registers** — `MRS` reads CPSR (mode leak); `CPS` is *ignored* in user mode; `LDM (3)` is *unpredictable* in user mode
- **Data-processing "S" variants** — replace CPSR with SPSR for exception return; unpredictable in user mode
- **Load/store with translation** (`LDRT` , `STRT`) — use user-mode permissions from a privileged mode; behave as ordinary accesses in user mode, so a de-privileged guest kernel misses expected faults

Many are defined as **unpredictable** rather than trapping — silence, not a trap.

Trade-Offs

Because MIPS, x86-32, and ARM all violate the theorem, **no** hypervisor could meet equivalence, safety, *and* performance by trap-and-emulate alone. Each system gave up something different.

	Disco	VMware Workstation	Xen	KVM for ARM
Architecture	MIPS	x86-32	x86-32	ARMv5
Hyp type	Type-1	Type-2 (§4.2.4)	Type-1 with dom0 (§4.4)	Type-2 (§4.5)
Equivalence	Requires modified kernel	Binary-compatible with selected kernels	Required modified (paravirtualized) kernels (§4.3)	Required modified (lightweight paravirtualized kernels (§4.5)
Safety	Via de-privileged execution using strictly virtualized resources	Via dynamic binary translation; isolation achieved via segment truncation	Via de-privileged execution with safe access to physical names	Via de-privileged execution using strictly virtualized resources
Performance	Via localized kernel changes and L2TLB (§4.1.2)	By combining direct execution (or applications) with adaptive dynamic binary translation (§4.2.3)	Via paravirtualization of CPU and IO interactions	Via paravirtualization of CPU and IO interactions

Virtualization without Architectural Support

VMware Workstation (1999): Goals and Criteria

First VMM for 32-bit x86. Commercial context dictated the architecture: closed-source guests ⇒ **recompiling the kernel was not an option**; a disaggregated x86 industry ⇒ virtualization had to slot in **without changing existing hardware or software**. So: **pure virtualization** for unmodified guests, as a **type-2** hypervisor on Linux and Windows.

Equivalence: x86-32's 17 sensitive-unprivileged instructions rule out trap-and-emulate. Solution: **direct execution** whenever possible + **dynamic binary translation (DBT)** when required.

Safety: the hardware is configured by the VMM; isolation relies heavily on **segment truncation**. Only the subset of x86-32 needed by supported guests was implemented — unsupported requests (e.g. code at `%cpl=1` or `2`) simply abort.

Performance: run relevant workloads at near-native speed — at worst, like the *immediately prior* CPU generation without virtualization.

Dynamic Binary Translation

DBT is an efficient form of emulation. It converts guest x86 machine code into modified x86 machine code that can execute safely under the hypervisor

Rather than interpret instruction by instruction, DBT

- Compiles a group of instructions into a fragment of executable code stored in a **translation cache (TC)**
- Classic optimizations: **chaining** (direct jumps between compiled fragments)
- The processor natively executes the translated sequence, not the original VM instructions
- DBT performance is very sensitive to hardware configuration: slowdown might come from **MMU emulation**
- DBT runs in the *same address space* as the code it emulates $\Rightarrow$ the VMM must share the VM's address space **safely and invisibly**

Choosing the Execution Engine

VMware's two key insights:

1. Direct execution covers *most* of X86, in particular **applications**
2. With careful use of segment protection, **system code under DBT** also runs near native speed

The algorithm is `0(1)` — it inspects only current register state, makes **no assumptions about future instructions**, and compiles to a handful of assembly instructions.

Overall: **DBT for the guest kernel, DE for applications.**

Input: Current state of the virtual CPU
Output: True if the direct execution subsystem may be used;
 False if binary translation must be used instead

```

1: if !cr0.pe then
2:   return false;
3: end if
4: if eflags.v8086 then
5:   return true
6: end if
7: if (eflags.iopl ≥ cpl) || (!eflags.if) then
8:   return false;
9: end if
10: for all seg ← (cs, ds, ss, es, fs, gs) do
11:   if “seg is not shadowed” then
12:     return false;
13:   end if
14: end for
15: return true
  
```

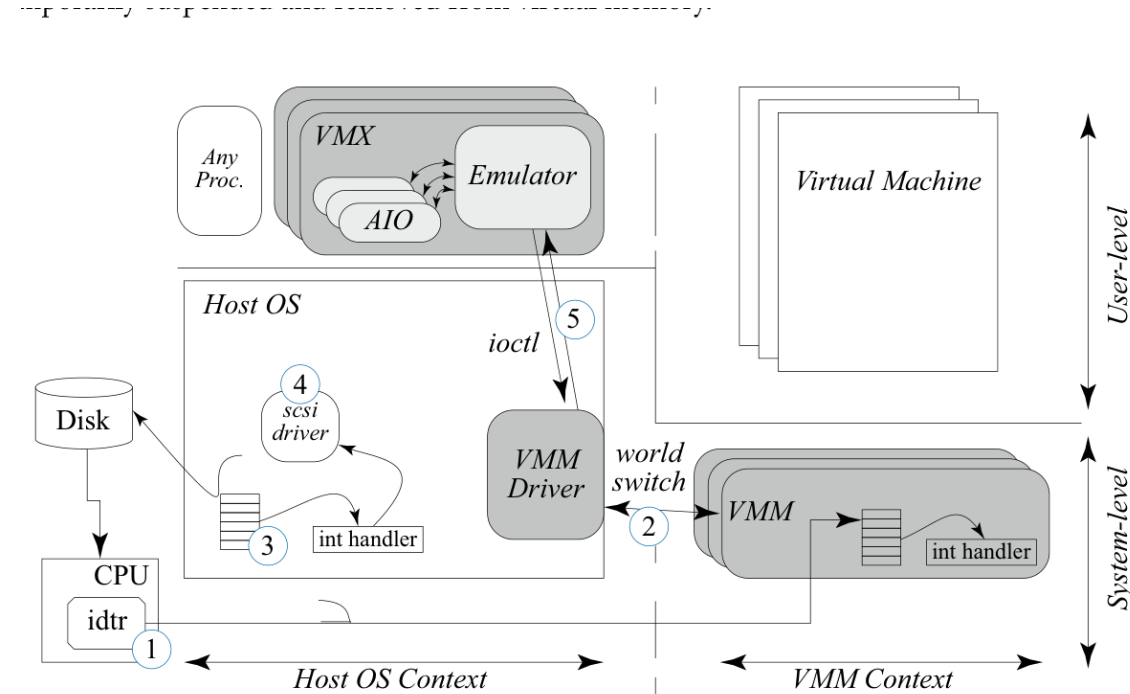
Reading the Selection Algorithm

- **Lines 1–3:** `!cr0.pe` $\Rightarrow$ `real` or `system management` mode $\Rightarrow$ **DBT** (e.g., the BIOS)
- **Lines 4–6:** `v8086` guests can always use hardware `v8086` mode $\Rightarrow$ **DE** (MS-DOS under Windows 95)
- **Lines 7–9:** if the VM can control the **interrupt flag** $\Rightarrow$ **DBT**. Covers guest kernel (`cp1=0`) and the rare user process using `iopl(2)`
- **Lines 10–14:** if the hidden state of any of the six segment descriptor registers is not recoverable from memory $\Rightarrow$ **DBT** (an obscure corner case needed for Windows 95)
- Otherwise $\Rightarrow$ **DE**, notably user level where interrupts can't be disabled

Host Operating System

VMware Workstation is type-2 but the VMM has **total control of the CPU** while a VM runs. Three components:

- **VMX**: user-level process: UI, host OS interaction, **I/O emulation**
- **VMM Driver**: a small kernel device driver
- **VMware VMM**: runs at system level, in a *disjoint* environment where the host OS has been temporarily suspended and removed from virtual memory



The World Switch

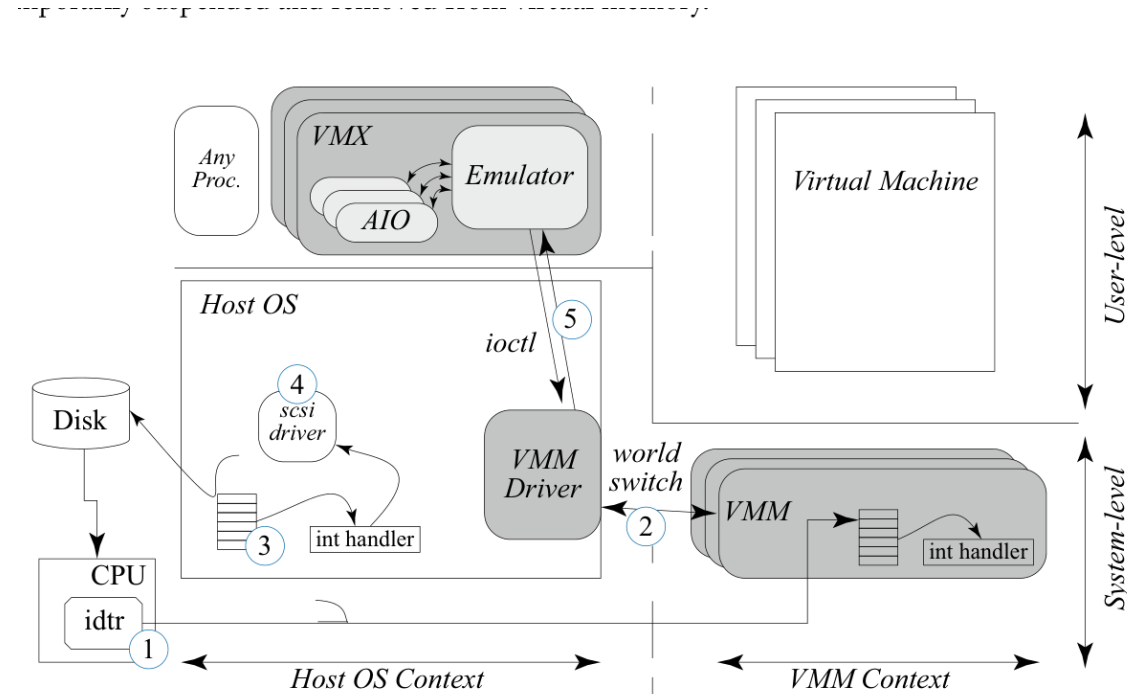
Like a context switch, but far broader in scope: it must save/restore **everything** — segment descriptor tables, interrupt descriptor tables — across *disjoint* address spaces.

Early implementation: a symmetric **45-instruction** sequence with interrupts disabled.

Example: A Disk Interrupt During VM Execution

1. Hardware interrupts the VM; `%idtr` points to a **VMware VMM** interrupt handler
2. The VMM cannot handle I/O $\Rightarrow$ it initiates a **world switch** back to the host OS context
3. The **VMM Driver** completes the switch and **re-issues the same interrupt in software**
4. The host OS handles it as if it had occurred normally — calling, e.g., the disk handler
5. Execution resumes in the VMM Driver, which the host OS thinks is merely servicing an `ioctl`; control returns to VMX, which processes asynchronous I/O (AIO) and issues the next `ioctl` to resume the VM

The host OS is **entirely oblivious** to the VMware VMM.



Xen — The Paravirtualization Alternative

VMware Workstation was an explicit trade of *equivalence for performance*: DBT and shadow paging performed poorly on many microbenchmarks and system-intensive workloads, and added atypical complexity.

Paravirtualization sacrifices certain aspects of the *equivalence* property in exchange for efficiency or scalability.

- Term introduced by **Denali** (2002), popularized by **Xen** (2003), the most influential open-source **type-1** hypervisor for x86
- Original x86-32 Xen simply **undefines all 17 non-virtualizable instructions**
- Replaces them with **hypercalls** plus additional memory-mapped system registers
- With this augmented ISA, Xen meets the Popek/Goldberg criteria and is a genuine **trap-and-emulate** hypervisor using **direct execution only**

The Xen Paravirtualized Interface

Xen changes the exposed ISA

	Memory Management
Segmentation	Cannot install fully privileged segment descriptors and cannot overlap with the top end of the linear address space.
Paging	Guest OS has direct read access to hardware page tables, but updates are batched and validated by the hypervisor. A domain may be allocated discontinuous machine (aka host-physical) pages.
	CPU
Protection	Guest OS must run at a lower privilege level than Xen.
Exceptions	Guest OS must register a descriptor table for exception handlers with Xen. Aside from page faults, the handler remains the same.
System calls	Guest OS may install a “fast” handler for system calls, allowing direct calls from an application into its guest OS and avoiding indirection through Xen on every call.
Interrupts	Hardware interrupts are replaced with a lightweight event system.
Time	Each guest OS has a timer interface and is aware of both “real” and “virtual” time.

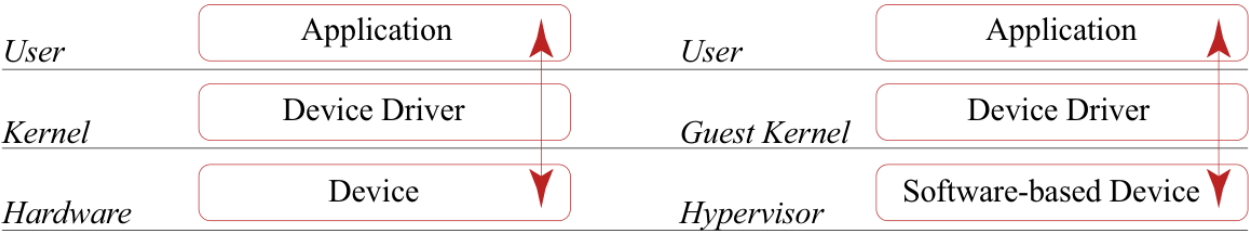
I/O Virtualization

Benefits

I/O virtualization **decouples** virtual I/O activity from physical I/O activity, letting the hypervisor interpose on every operation.

Benefits:

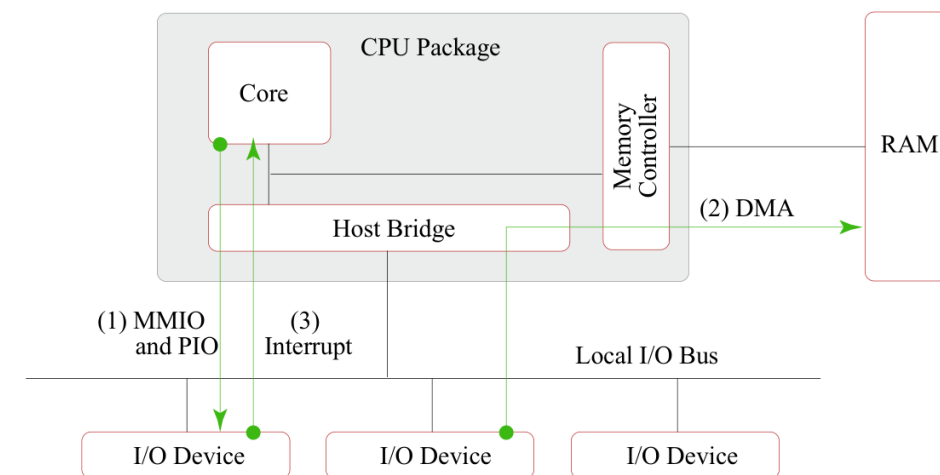
- **Encapsulation:** the full VM state, including state of I/O devices, can be saved as an image
- **Portability & live migration:** resume on a server with *different* devices; the VM need not even stop
- **Consolidation:** map many virtual devices onto fewer physical ones
- **Aggregation:** unify several devices into one superior virtual device (load balancing, failure masking)
- **New features:** snapshots, record-replay, etc.
- **Memory optimizations:** overcommitment, demand paging, swapping, copy-on-write, page migration, THP



Physical I/O: Three Interaction Modes

Devices interact with CPU and memory in exactly three ways:

- **Port-Mapped I/O (PIO) / Memory-Mapped I/O (MMIO)** — cores talk to devices. PIO uses a separate 16-bit port space via `IN / OUT` ; MMIO uses ordinary loads/stores to addresses assigned at startup
- **Direct Memory Access (DMA)** — the device accesses memory directly, without core involvement; the core initiates and is notified on completion
- **Interrupts** — devices notify cores asynchronously. Vector → IDT entry; per-core **Local Advanced Programmable Interrupt Controller (LAPIC)** manages delivery



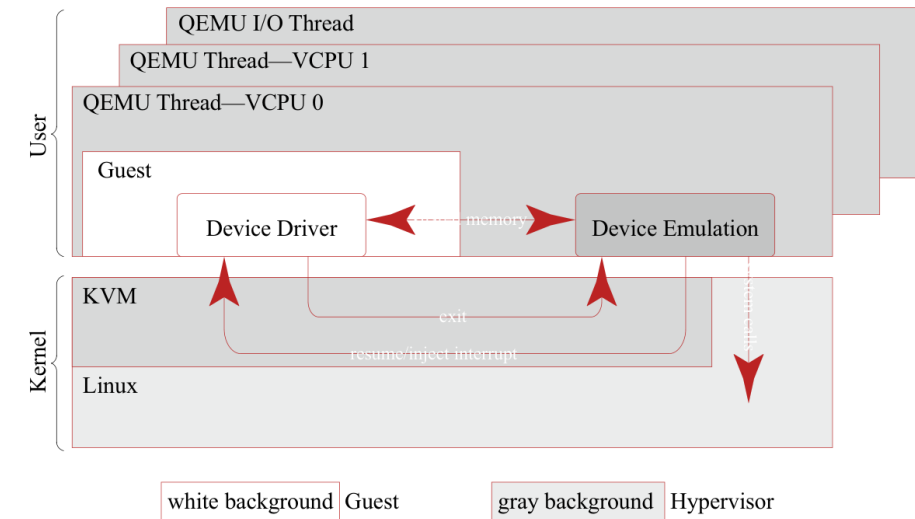
I/O Virtualization w/o HW support

I/O Emulation (Full Virtualization)

Since PIO, MMIO, DMA, and interrupts are the *only* interactions, the hypervisor can sustain the whole illusion by:

1. Arranging for every guest PIO and MMIO to **trap**
 - MMIO: map the pages non-present or read-only
 - PIO: configure the VMCS to exit on those ports
2. Responding as a real device would — **injecting interrupts** and reading/writing guest-physical memory as if performing DMAs

In KVM/QEMU: each VM is a QEMU process; VCPUs are threads with two contexts (guest and host); each virtual device gets an **I/O thread** for asynchronous work.



I/O Paravirtualization

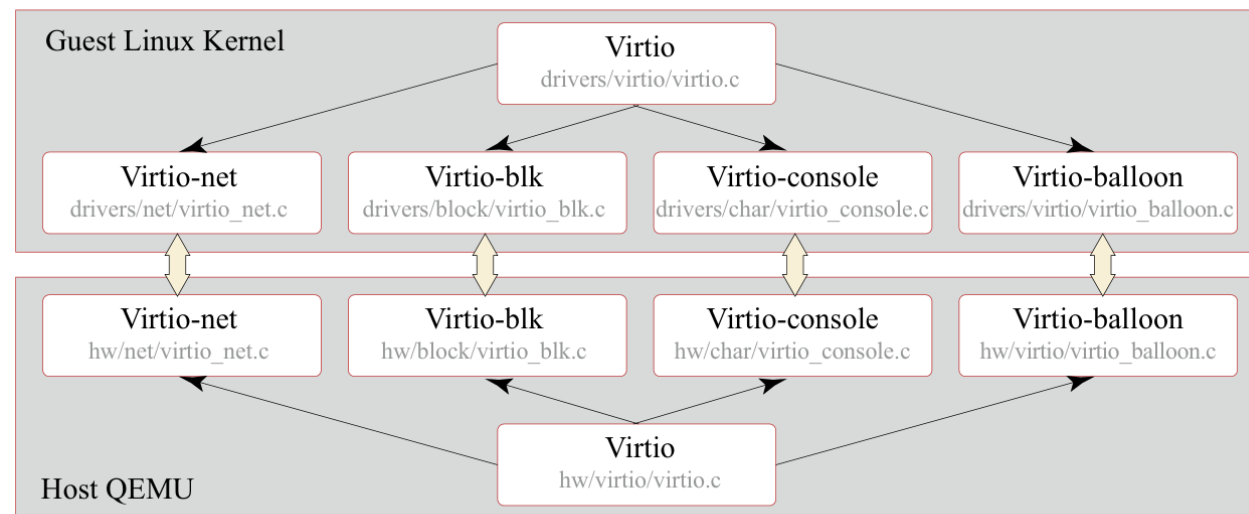
Guest and host agree on a **virtual device specification designed to minimize exits**. The guest installs a driver compatible only with its hypervisor instead of real hardware.

Virtio: Framework of paravirtual I/O devices of KVM/QEMU. The central construct is the **virtqueue**:

- Emulated devices exit **implicitly** on every MMIO/PIO, severely degrading performance. virtqueue guests exit only when they *choose to*, via `virtqueue_kick` API call

Two Execution modes to further optimize performance:

- NO_INTERRUPT** : guest tells host to stop delivering interrupts, hence reducing interrupts
- NO_NOTIFY** : host tells guest to stop kicking, reducing exits.



I/O Virtualization w/ HW support

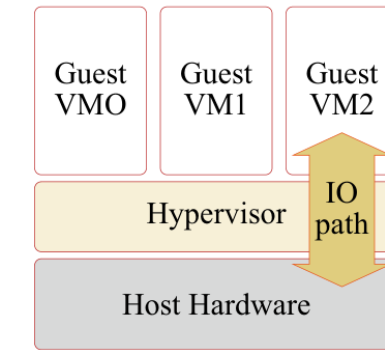
Direct Device Assignment

Give up interposition, gain performance: dedicate a physical device **d** exclusively to VM **v**, so **v** drives it with no software intermediary.

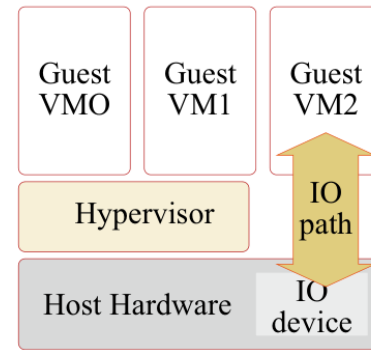
Two serious problems with **naïve** assignment:

- **Scalability:** spare physical devices are inherently scarce; a server hosts far more VMs than devices
- **Safety:** if **v** controls **d**, it can program **d** to DMA to *any* physical address, including other VMs' and the hypervisor's memory. Isolation is effectively gone

Hardware fixes both: **IOMMU** for security, **SRIOV** for scalability. Interrupts remain the last major overhead $\Rightarrow$ exitless/posted interrupts.



(a) Emulation/paravirtualization

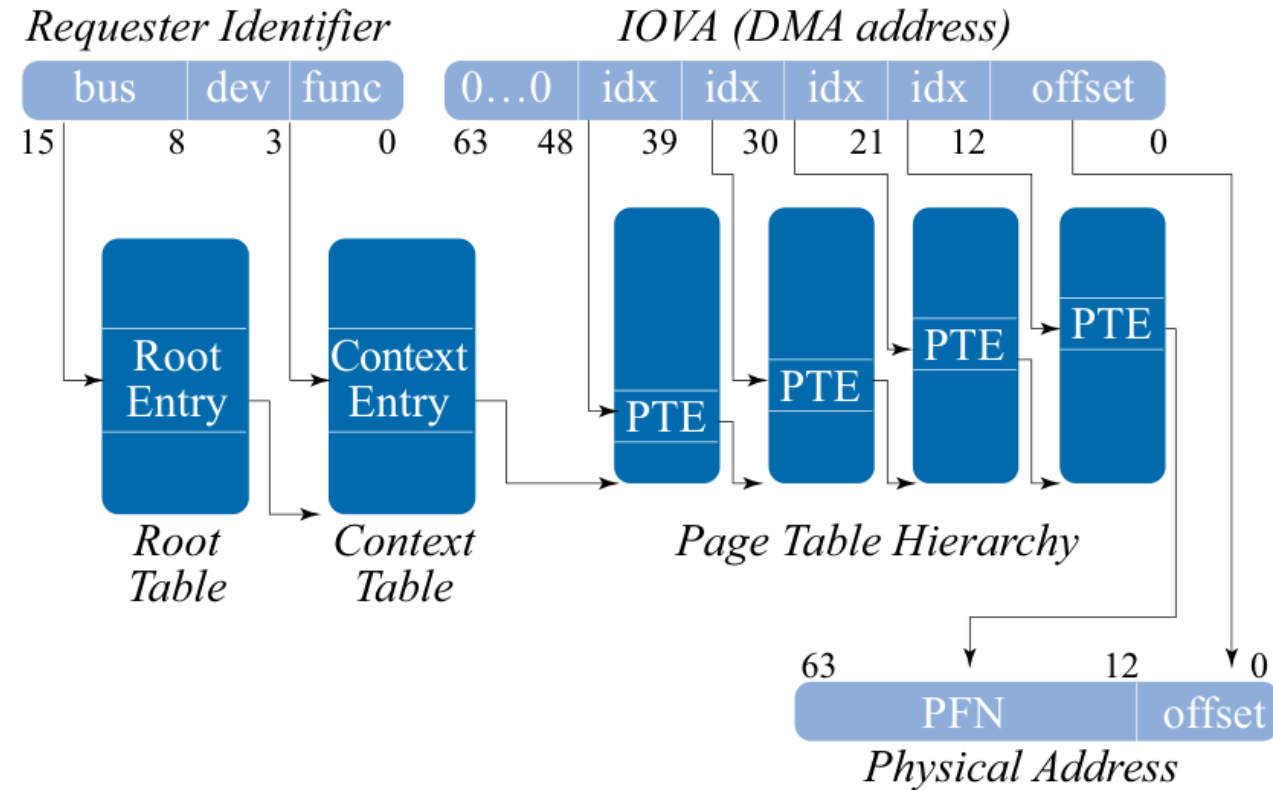


(b) Direct device assignment

I/O Memory Management Unit (IOMMU)

Two engines:

- **DMAR** (DMA remapping) translates **I/O virtual addresses (IOVAs)** to physical addresses via hypervisor-set page tables
 - Translation is invalid or permission mismatch: translation fails
- **IR** (interrupt remapping) translates interrupt vectors fired by devices.



Interrupt Remapping

Without IR, an assigned device is a **privilege escalation vector**:

- 1. The guest's config space is **emulated**, so the hypervisor maps guest vector 30 ↔ physical vector 50 internally
- 2. Device fires vector 50 → passes through IOMMU unchanged → LAPIC → exit → hypervisor injects vector 30

The hole: MSIs are just **memory writes** to `0xFEEx_xxxx`. A VM controlling a device can make it **DMA any vector** into the interrupt space, and without IR the IOMMU cannot tell a genuine MSI from a forged one.

With IR: MSI (message signaled interrupt) registers hold an **IRindex** into the IR table (set only by the hypervisor). Each entry carries target LAPIC, vector, and the device's BDF (Bus–Device–Function, the address identifying a PCI/PCIe device function.) for anti-spoofing, so vectors cannot be forged.

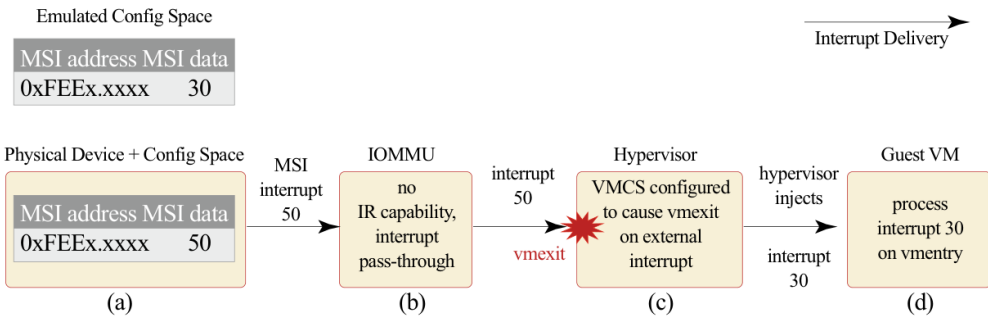
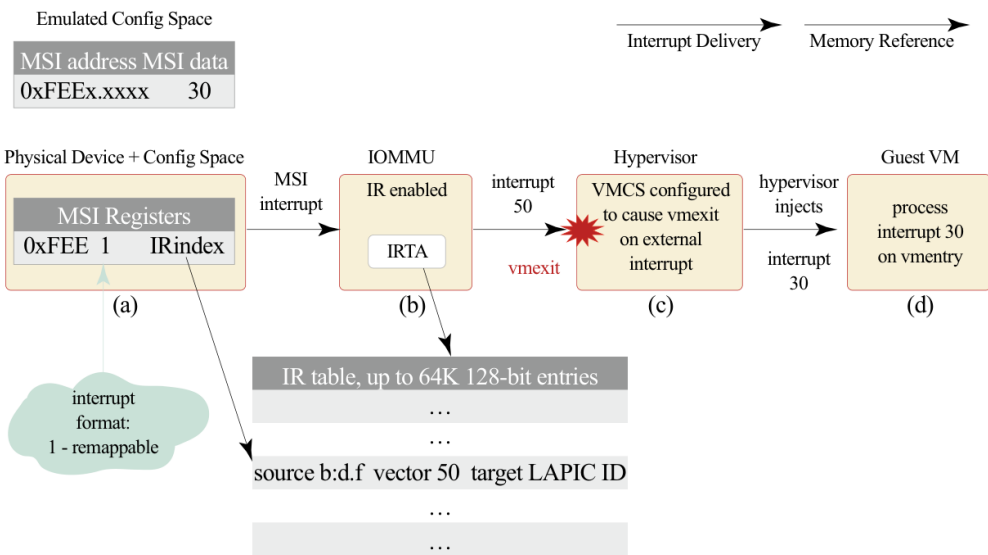


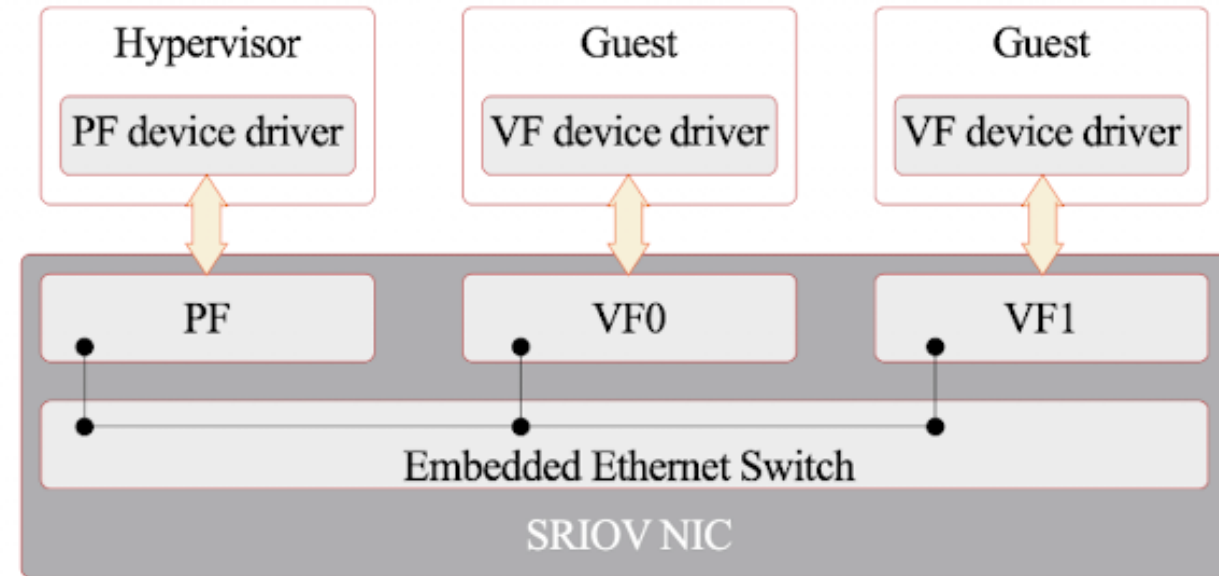
Figure 6.16: MSI interrupt delivery without interrupt remapping support.



SRIOV: Making Assignment Scalable

An **SRIOV-capable device self-virtualizes in hardware**, presenting multiple instances of itself, each assignable to a different VM.

- **PF (Physical Function)**: a standard PCIe function; the host manages it and can allocate/deallocate/configure VFs
- **VF (Virtual Function)**: a lightweight function with a *limited* config space (no independent power management, cannot manage other VFs). Assigned to a VM, it gives direct I/O with the hypervisor **out of the data path**



The Three Virtual I/O Models

Virtual I/O Model	Emulation	Paravirtualization	Device Assignment
Portable, no host-specific software	✓	✗	✓
Interposition and memory optimizations	✓	✓	✗
Performance	worse	better	best

- **Emulation:** portable, interposable, slowest
- **Paravirtualization:** interposable and much faster, but needs hypervisor-specific guest drivers
- **Direct assignment:** fastest and portable, but **forfeits interposition**

Summary

- **Virtualization** = layering + enforced modularity, built from multiplexing, aggregation, and emulation; a **hypervisor** is distinguished from a simulator by the *performance* criterion
- **Popek/Goldberg**: a VMM exists when sensitive $\subseteq$ privileged. MIPS, x86-32, and ARM all violated it — via direct physical-memory access, location-sensitivity, and control-sensitivity
- **VMware Workstation**: preserved equivalence with **adaptive dynamic binary translation**, **segment truncation**, the **world switch**, **memory tracing**, and **shadow page tables**
- **Xen**: traded equivalence for efficiency — **hypercalls**, exposed host-physical names, `dom0` device drivers
- **I/O virtualization**: emulation $\rightarrow$ **virtio** (batching, `NO_NOTIFY` / `NO_INTERRUPT`) $\rightarrow$ **direct assignment**, made safe by the **IOMMU + interrupt remapping**, scalable by **SRIOV**, and exit-free by **ELI** and **posted interrupts**

Questions?

Reading: Bugnion, Nieh, and Tsafrir, *Hardware and Software Support for Virtualization*, Ch. 1, 2, 3.2–3.3, 6.